



Deep Dive into Large Language Models

A comprehensive exploration of how LLMs work — from architecture and attention mechanisms to model types and selection strategies.

What Are Large Language Models?

Core Definition

LLMs are deep learning models trained on massive text datasets to understand, generate, and reason over natural language. They are built on **Transformer architecture** and trained using **self-supervised learning**.

Key Capabilities

- Natural Language Understanding
- Text & Code Generation
- Reasoning & Question Answering
- Summarization & Translation
- Agentic Task Planning



AI Assistants

Conversational AI systems and autonomous agents



Code Copilots

Intelligent coding assistance and generation



Document Analysis

Summarization, extraction, and comprehension

Notable LLMs Today

GPT-4 / GPT-4o

OpenAI's flagship models with broad reasoning and multimodal capabilities

Claude

Anthropic's safety-focused model with large context windows

Gemini

Google's multimodal LLM with up to 1M+ token context

LLaMA & Mistral

Open-weight models enabling local and private deployments



Transformer Architecture: The 2026 Blueprint

"Attention is All You Need" — Vaswani et al., 2017

The original Transformer (2017) remains foundational, but modern LLMs have evolved significantly — adopting Decoder-Only architectures, Mixture-of-Experts (MoE) scaling, and sub-quadratic attention.

The Transformer is the neural network architecture powering modern LLMs. Unlike earlier models, Transformers process text **in parallel** rather than sequentially, enabling large-scale training.

1

Decoder-Only Architecture

Pre-Layer Normalization for training stability; standard in GPT-5, Gemini 2.5

2

Mixture-of-Experts (MoE)

Sparse routing activates only a fraction of parameters per token (e.g., 35B active / 480B total), reducing inference FLOPs

3

Multi-Head Self-Attention

Models relationships between all tokens; now enhanced with FlashAttention-3

4

Feed Forward Network

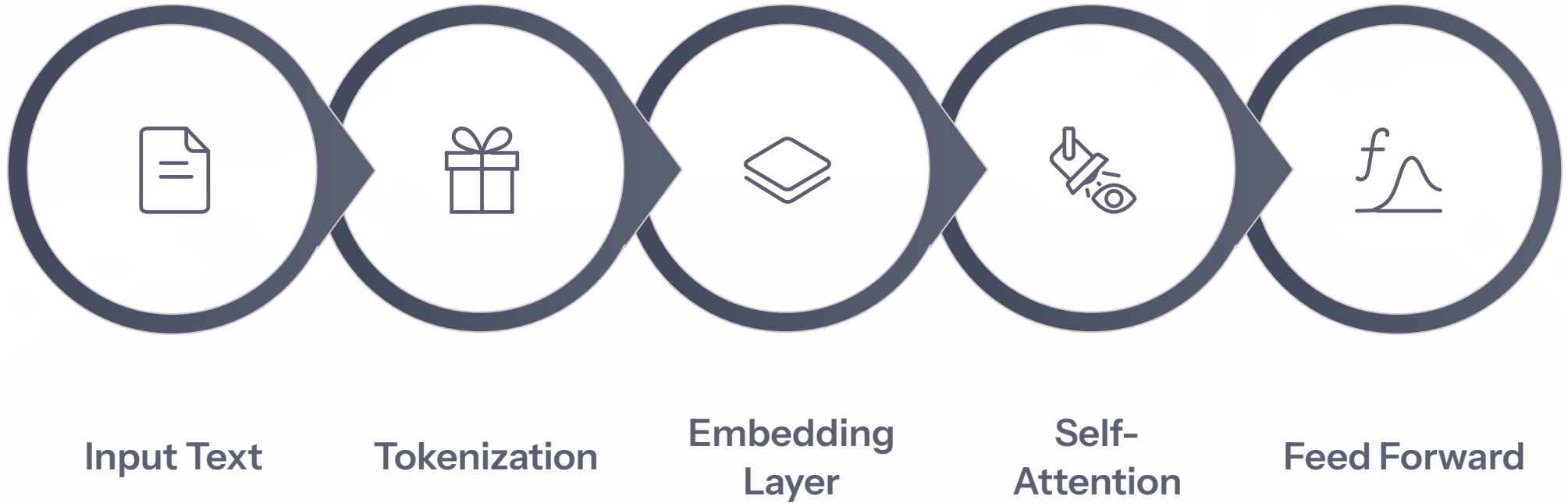
Transforms attended representations; replaced by expert sub-networks in MoE models

5

Dual-Branch Fusion

Parallel multimodal streams (audio/visual) merged via cross-attention (e.g., Seedance 2.0)

Transformer Pipeline



Each step transforms raw text into a probability distribution over possible next tokens — the core mechanism behind LLM text generation.

Tokenization & Embeddings

What is Tokenization?

Tokenization splits text into smaller units — words, subwords, or characters — that the model can process.

❏ **Example:** "AI agents will transform automation." → AI
| agents | will | transform |
automation

Token Embeddings

Each token becomes a high-dimensional vector encoding its semantic meaning. This enables the famous analogy:

❏ **King - Man + Woman $\approx$ Queen**

Embeddings allow LLMs to understand relationships, capture meaning, and reason across context.

AN
INEATIONCE
INNOVATION
PROGRESS
FUTURE
TIDACS AND
FOU

The Attention Mechanism: 2026 Advances

Attention allows the model to determine **which words matter most** — assigning weights rather than treating every word equally. In 2026, key innovations have pushed beyond the original softmax bottleneck.

📌 **Example:** "The robot picked up the battery because *it* was low." — Attention resolves that "**it**" → **battery**.

Attention Formula

Computed using three vectors: **Query (Q)**, **Key (K)**, and **Value (V)**.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right) V$$

Now accelerated with **FlashAttention-3**, bypassing the $O(n^2)$ complexity bottleneck.

2026 Attention Innovations

- **Multi-Head Latent Attention (MHSA):** Restores expressivity in linear attention with "Retriever" and "Mixer" head specializations.
- **Token-Level Sparsification:** Dynamic selection of tokens worth attending to per layer.
- **RoPE + YaRN:** Rotary Positional Embeddings extended via YaRN for far-beyond-training-length extrapolation.

Context Window: The 2026 Frontier

The context window defines how much a model can 'remember' at once. By 2026, windows have scaled dramatically — but size alone doesn't guarantee quality.

Model	Context Window
GPT-4o	128K tokens
Claude 3.7	200K tokens
Gemini 2.5 Pro	1M+ tokens
Llama 4 Scout	10M tokens (entire code repositories)

The Quality vs. Quantity Paradox

- "Lost-in-the-Middle": Attention weight decays in the center of long sequences
- Context Rot: Performance degrades beyond ~100K tokens; models repeat historical actions
- KV Cache Bloat: Memory overhead grows linearly with context length

2026 Solutions

- Context Pruning: Up to 95% compression of irrelevant tokens
- Semantic Caching: Reuse of previously computed context representations
- State Space Models (Mamba-2): Linear-time inference with constant memory, solving KV Cache bloat
- RAG + Vector Databases: Retrieval-augmented approaches for selective context injection



Types of AI Models: Q1 2026

1

Reasoning (Thinking) Models

Optimized for multi-step logic, complex math, and autonomous bug-fixing. Examples: OpenAI o1, DeepSeek-R1. Use cases: code debugging, scientific reasoning, autonomous agents.

2

Multimodal Sparse MoE

Unified processing of text, 1-hour+ video, and real-time audio via sparse Mixture-of-Experts routing. Examples: Gemini 2.5 Pro, GPT-4o. Use cases: video understanding, cross-modal reasoning.

3

Long-Context Data Processors

Designed for repository-scale inputs up to 10M tokens. Examples: Llama 4 Scout. Use cases: large codebase migration, legal/medical document audits.

4

Agentic Coding Models

Autonomous software engineering via dedicated Agent SDKs. Examples: Claude 4.5 Sonnet. Use cases: end-to-end feature development, test generation, CI/CD integration.

AI Model Selection Strategy

Choosing the right model depends on multiple factors. The architecture below illustrates a typical end-to-end AI pipeline.

1 Task Complexity

Simple tasks → SLM. Complex reasoning → Large LLM.

2 Latency Requirements

Real-time applications require faster, lighter models.

3 Deployment Environment

Cloud inference, on-premise, or edge devices each have different constraints.

4 Cost Constraints

Larger models carry higher inference costs — balance capability with budget.

Example Pipeline

01

User Query

02

Embedding Model

03

Vector Database

04

LLM Reasoning

05

Agent Tool Execution

06

Final Response